

UNIwersYTET ŚLĄSKI

WNŚT

Informatyka

Projekt zaliczeniowy

Rolling Village

Webowa adaptacja gry planszowej

Projekt systemu - aplikacje multimedialne

Antoni Załupka
Bartłomiej Marzec

Grudzień 2025

Spis treści

1	Wstęp	2
2	Stack technologiczny	2
2.1	Framework i środowisko uruchomieniowe	2
2.2	Biblioteki interfejsu użytkownika	2
2.3	System stylowania	3
2.4	Narzędzia deweloperskie i wdrożeniowe	3
3	Architektura aplikacji	4
3.1	Struktura katalogów projektu	4
3.2	Wzorzec architektoniczny	5
3.3	System routingu aplikacji	5
4	Implementacja logiki gry	6
4.1	Klasa silnika gry RollingVillage	6
4.2	System rzutu kośćmi i mapowanie wierszy	6
4.3	Typy budynków i algorytm punktacji	7
4.4	Mechanizm cofania i ponawiania akcji	7
5	Komponenty interfejsu użytkownika	7
5.1	Strona główna i formularz inicjalizacji	7
5.2	Widok gry i komponenty planszy	8
5.3	Responsywność interfejsu	8
5.4	Komponenty biblioteki shadcn/ui	9
6	Problemy i rozwiązania	9
6.1	Reaktywność stanu gry w kontekście React	9
6.2	Implementacja pętli gry z wykorzystaniem requestAnimationFrame	9
6.3	Algorytm wyszukiwania alternatywnych kolumn	10
7	Podsumowanie	10
8	Bibliografia	12

1 Wstęp

Niniejsze sprawozdanie przedstawia proces projektowania i implementacji aplikacji webowej stanowiącej cyfrową adaptację gry planszowej Rolling Village. Projekt został zrealizowany z wykorzystaniem nowoczesnych technologii frontendowych, ze szczególnym uwzględnieniem frameworka Next.js oraz biblioteki React. Głównym celem przedsięwzięcia było stworzenie interaktywnej, responsywnej aplikacji umożliwiającej rozgrywkę w trybie jednoosobowym, z zachowaniem oryginalnych mechanik gry planszowej.

Rolling Village to gra strategiczna, w której gracze wcielają się w rolę architektów budujących własne miasto. Mechanika opiera się na rzutach kośćmi, które determinują dostępne budynki oraz lokalizacje na planszy. Gracz musi podejmować decyzje dotyczące optymalnego rozmieszczenia budynków, maksymalizując punkty końcowe. Cyfrowa wersja gry zachowuje wszystkie te elementy, jednocześnie oferując intuicyjny interfejs użytkownika oraz automatyzację obliczeń punktowych.

Aplikacja została zaprojektowana z myślą o rozszerzalności, co przejawia się w przygotowanej strukturze katalogów dla trybu wieloosobowego. Sprawozdanie szczegółowo omawia zastosowany stack technologiczny, architekturę systemu, implementację kluczowych funkcjonalności oraz napotkane wyzwania techniczne wraz z przyjętymi rozwiązaniami.

2 Stack technologiczny

2.1 Framework i środowisko uruchomieniowe

Fundamentem aplikacji jest framework Next.js w wersji 15.5.4, reprezentujący nowoczesne podejście do budowania aplikacji React. Wybór tego frameworka podyktowany został jego zaawansowanymi możliwościami, w szczególności mechanizmem App Router wprowadzonym w wersji 13, który oferuje intuicyjny system routingu oparty na strukturze katalogów. Next.js zapewnia również wbudowane wsparcie dla renderowania po stronie serwera (SSR) oraz generowania statycznych stron (SSG), choć w przypadku niniejszego projektu wykorzystano głównie renderowanie po stronie klienta ze względu na interaktywny charakter gry.

Aplikacja wykorzystuje React w wersji 19.1.0, stanowiącej najnowsze wydanie tej biblioteki. React 19 wprowadza szereg optymalizacji wydajnościowych oraz nowe hooki, które zostały wykorzystane przy implementacji logiki gry. Całość kodu źródłowego została napisana w języku TypeScript w wersji 5, co zapewnia statyczne typowanie i znacząco redukuje ryzyko wystąpienia błędów w czasie wykonania. Typowanie obejmuje zarówno komponenty React, jak i klasę silnika gry, definiując precyzyjne interfejsy dla wszystkich struktur danych.

Środowisko deweloperskie wykorzystuje Turbopack, eksperymentalny bundler zintegrowany z Next.js, który oferuje znacznie szybsze czasy kompilacji w porównaniu do tradycyjnego Webpacka. Konfiguracja projektu obejmuje również ESLint do statycznej analizy kodu oraz Docker do konteneryzacji aplikacji zarówno w środowisku deweloperskim, jak i produkcyjnym.

2.2 Biblioteki interfejsu użytkownika

Warstwa prezentacji aplikacji została zbudowana w oparciu o system komponentów shadcn/ui, wykorzystujący styl wizualny określany jako `new-york`. Wybór ten podyktowany

został eleganckim wyglądem komponentów oraz ich pełną dostępnością (accessibility). W odróżnieniu od tradycyjnych bibliotek komponentów, shadcn/ui dostarcza kod źródłowy komponentów bezpośrednio do projektu, co umożliwia ich pełną personalizację bez konieczności nadpisywania stylów zewnętrznej biblioteki.

Fundamentem komponentów shadcn/ui są prymitywy Radix UI, zapewniające poprawną obsługę dostępności oraz interakcji użytkownika. W projekcie wykorzystano w szczególności komponent `Dialog` w wersji 1.1.15, służący do wyświetlania modalnych okien dialogowych na urządzeniach mobilnych, oraz `Slot` w wersji 1.2.4, umożliwiający kompozycję komponentów poprzez przekazywanie właściwości do elementów potomnych.

Ikony w aplikacji pochodzą z biblioteki Lucide React w wersji 0.548.0, oferującej obszerny zestaw spójnych wizualnie ikon wektorowych. W kontekście gry planszowej szczególnie istotne okazały się ikony reprezentujące kostki do gry, wykorzystywane w interfejsie rzutu kośćmi.

2.3 System stylowania

Stylowanie aplikacji opiera się na frameworku Tailwind CSS w wersji 4, reprezentującym podejście utility-first do pisania stylów CSS. Podejście to polega na kompozycji małych, jednozadaniowych klas bezpośrednio w znacznikach JSX, co eliminuje konieczność tworzenia osobnych plików CSS i znacząco przyspiesza proces prototypowania interfejsu.

Zarządzanie wariantami komponentów realizowane jest przy użyciu biblioteki `class-variance-authority` w wersji 0.7.1, która umożliwia definiowanie różnych stanów wizualnych komponentu w sposób deklaratywny i typowany. Komponent `Button` wykorzystuje tę bibliotekę do definiowania siedmiu wariantów: `default`, `destructive`, `outline`, `secondary`, `ghost` oraz `link`.

Łączenie klas CSS w sposób bezpieczny i eliminujący konflikty realizowane jest przez kombinację bibliotek `clsx` w wersji 2.1.1 oraz `tailwind-merge` w wersji 3.3.1. Funkcja pomocnicza `cn()`, zdefiniowana w pliku `src/lib/utils.ts`, stanowi centralny punkt łączenia klas Tailwind w całej aplikacji. Animacje interfejsu użytkownika zostały zaimplementowane przy użyciu biblioteki `tw-animate-css` w wersji 1.4.0, oferującej zestaw predefiniowanych animacji kompatybilnych z Tailwind CSS.

2.4 Narzędzia deweloperskie i wdrożeniowe

Proces budowania aplikacji wykorzystuje Turbopack, co znajduje odzwierciedlenie w skryptach zdefiniowanych w pliku `package.json`. Polecenie `npm run dev` uruchamia serwer deweloperski z flagą `-turbopack`, natomiast `npm run build` kompiluje aplikację produkcyjną. Serwer produkcyjny nasłuchuje na porcie 3020, co zostało skonfigurowane w skrypcie `start`.

Konteneryzacja aplikacji została zrealizowana przy użyciu Docker, z oddzielnymi konfiguracjami dla środowiska deweloperskiego (`docker-compose.yml`) oraz produkcyjnego (`docker-compose.prod.yml`). Plik `Dockerfile` definiuje wieloetapowy proces budowania obrazu, optymalizując rozmiar finalnego kontenera poprzez separację zależności deweloperskich od produkcyjnych.

Statyczna analiza kodu realizowana jest przez ESLint, którego konfiguracja znajduje się w pliku `eslint.config.mjs`. Narzędzie to weryfikuje zgodność kodu ze zdefiniowanymi regułami stylistycznymi oraz wykrywa potencjalne problemy przed uruchomieniem aplikacji.

3 Architektura aplikacji

3.1 Struktura katalogów projektu

Organizacja kodu źródłowego odzwierciedla konwencje przyjęte w ekosystemie Next.js z wykorzystaniem App Router. Katalog główny `src/` zawiera cztery podkatalogi o ściśle określonych odpowiedzialnościach. Katalog `app/` stanowi rdzeń systemu routingu, gdzie struktura folderów bezpośrednio przekłada się na ścieżki URL aplikacji. Katalog `components/` grupuje wielokrotnie używane komponenty React, podzielone tematycznie na podkatalogi. Katalog `game/` zawiera logikę biznesową gry, całkowicie niezależną od warstwy prezentacji. Katalog `types/` przechowuje globalne definicje typów TypeScript, a `lib/` zawiera funkcje pomocnicze wykorzystywane w całej aplikacji.

Poniżej przedstawiono pełną strukturę katalogów projektu:

Listing 1: Struktura katalogów projektu

```
rolling-village/
|-- Dockerfile
|-- README.md
|-- components.json
|-- docker-compose.prod.yml
|-- docker-compose.yml
|-- eslint.config.mjs
|-- next-env.d.ts
|-- next.config.ts
|-- package.json
|-- postcss.config.mjs
|-- tsconfig.json
|-- document/
|   |-- main.tex
|-- public/
|   |-- game/
|       |-- building/
|       |-- ui/
|-- src/
|   |-- app/
|       |-- favicon.ico
|       |-- globals.css
|       |-- layout.tsx
|       |-- page.tsx
|       |-- api/
|           |-- rooms/
|       |-- play/
|           |-- multi/
|           |-- single/
|   |-- components/
|       |-- Board/
|           |-- Board.tsx
|           |-- GameState.tsx
|           |-- RemainingBuildings.tsx
|       |-- Building/
|           |-- Cell.tsx
```

```

|   |   |-- SelectBuildingMenu.tsx
|   |-- Score/
|   |   |-- Score.tsx
|   |-- ui/
|       |-- button.tsx
|       |-- Decorations.tsx
|       |-- dialog.tsx
|-- game/
|   |-- Dice.ts
|   |-- RollingVillage.ts
|-- lib/
|   |-- utils.ts
|-- types/
|   |-- types.d.ts

```

Struktura katalogów komponentów odzwierciedla logiczny podział interfejsu gry. Podkatalog `Board/` zawiera komponenty związane z planszą gry, w tym główny komponent `Board.tsx` renderujący siatkę oraz `GameState.tsx` odpowiedzialny za wyświetlanie stanu rozgrywki. Podkatalog `Building/` grupuje komponenty związane z budynkami, takie jak `Cell.tsx` reprezentujący pojedynczą komórkę planszy oraz `SelectBuildingMenu.tsx` służący do wyboru typu budynku. Podkatalog `Score/` zawiera komponent `Score.tsx` prezentujący punktację i informacje o rundzie. Podkatalog `ui/` przechowuje generyczne komponenty interfejsu pochodzące z `shadcn/ui` oraz niestandardowe elementy dekoracyjne.

3.2 Wzorzec architektoniczny

Aplikacja została zaprojektowana zgodnie z wzorcem Client-Side Game Engine, który zakłada ścisłą separację logiki gry od warstwy prezentacji. Centralnym elementem architektury jest klasa `RollingVillage`, zdefiniowana w pliku `src/game/RollingVillage.ts`, która enkapsuluje całą mechanikę gry w sposób całkowicie niezależny od frameworka React. Klasa ta liczy około 700 linii kodu i stanowi samodzielną jednostkę, którą można testować oraz wykorzystywać w izolacji.

Komponenty React pełnią wyłącznie rolę warstwy prezentacyjnej, odpowiedzialnej za renderowanie aktualnego stanu gry oraz przekazywanie akcji użytkownika do silnika gry. Komunikacja między warstwami odbywa się poprzez wywoływanie publicznych metod klasy `RollingVillage`, takich jak `build()`, `tick()` czy `setRollDice()`, oraz odczytywanie stanu gry za pomocą getterów, na przykład `getRemainingPlacements()`, `getScore()` czy `getRoundPhase()`.

Wzorzec ten zapewnia jednolite źródło prawdy (single source of truth) dla stanu gry, co eliminuje problemy związane z synchronizacją danych między komponentami. Każda zmiana stanu gry zachodzi wyłącznie poprzez metody klasy silnika, a komponenty jedynie reagują na te zmiany, renderując zaktualizowany interfejs.

3.3 System routingu aplikacji

Routing aplikacji wykorzystuje mechanizm App Router wprowadzony w Next.js 13, gdzie struktura katalogów w obrębie `src/app/` bezpośrednio definiuje dostępne ścieżki URL. Plik `layout.tsx` w katalogu głównym aplikacji definiuje wspólny układ strony, obejmujący konfigurację czcionek (Geist Sans i Geist Mono) oraz metadane dokumentu HTML.

Strona główna aplikacji, dostępna pod adresem root (/), renderowana jest przez komponent zdefiniowany w pliku `src/app/page.tsx`. Stanowi ona punkt wejścia dla użytkownika, prezentując formularz inicjalizacji gry w stylistyce wniosku o pozwolenie na budowę.

Tryb gry jednoosobowej dostępny jest pod ścieżką `/play/single`, której odpowiada plik `src/app/play/single/page.tsx`. Komponent ten zawiera pełną implementację widoku gry, włączając inicjalizację silnika, pętlę gry oraz renderowanie wszystkich elementów interfejsu.

Struktura katalogów przewiduje również ścieżkę `/play/multi/[id]` dla trybu wieloosobowego, gdzie `[id]` stanowi dynamiczny segment URL reprezentujący identyfikator pokoju gry. Katalog ten zawiera obecnie jedynie strukturę przygotowaną pod przyszłą implementację.

4 Implementacja logiki gry

4.1 Klasa silnika gry `RollingVillage`

Centralnym elementem logiki aplikacji jest klasa `RollingVillage`, zdefiniowana w pliku `src/game/RollingVillage.ts`. Klasa ta implementuje kompletny automat stanów gry, zarządzając przejściami między fazami rozgrywki oraz walidując akcje gracza. Konstruktor klasy przyjmuje parametry inicjalizacyjne, takie jak nazwa gracza i nazwa miasta, a następnie inicjalizuje planszę gry jako 30-elementową tablicę reprezentującą planszę o wymiarach 6 kolumn na 5 wierszy.

Gra przebiega przez trzy główne fazy, reprezentowane przez typ `GamePhase`: `setup` (konfiguracja początkowa), `main` (właściwa rozgrywka) oraz `gameover` (zakończenie gry). W ramach fazy głównej każda runda składa się z podfaz określonych typem `RoundPhase`: `build` (umieszczanie budynków na planszy), `bonus` (wybór budynku bonusowego, występujący co trzecią rundę) oraz `calculate` (obliczanie punktów dla wybranego wiersza).

Metoda `tick()` stanowi główną pętlę aktualizacji stanu gry, wywoływaną cyklicznie przez komponent React. Metoda ta sprawdza bieżący stan gry i automatycznie przeprowadza przejścia między fazami, gdy spełnione są odpowiednie warunki. Przykładowo, po zatwierdzeniu przez gracza rozmieszczenia budynków, metoda `tick()` automatycznie przechodzi do fazy obliczania punktów lub fazy bonusowej.

4.2 System rzutu kośćmi i mapowanie wierszy

Mechanika rzutu kośćmi została zaimplementowana w osobnej klasie `Dice`, zdefiniowanej w pliku `src/game/Dice.ts`. Klasa ta udostępnia statyczną metodę `roll()`, która generuje wynik rzutu dwiema sześciennymi kośćmi, zwracając obiekt zawierający wartości obu kostek oraz informację o typie dostępnego budynku.

Suma oczek na kościach determinuje wiersz planszy, w którym gracz może umieścić budynek. Mapowanie sum na wiersze zostało zdefiniowane w stałej `MAP_ROWS` wewnątrz klasy `RollingVillage`. Sumy 3 i 4 odpowiadają wierszowi 0, sumy 5 i 6 wierszowi 1, suma 7 wierszowi 2, sumy 8 i 9 wierszowi 3, a sumy 10 i 11 wierszowi 4. Szczególne przypadki stanowią sumy 2 oraz 12, które pozwalają graczowi na swobodny wybór dowolnego wiersza.

Wynik rzutu kośćmi określa również typ dostępnego budynku. Gdy obie kostki pokazują ten sam budynek, ale różne wartości, gracz otrzymuje możliwość postawienia fabryki

(**factory**). Natomiast dublet, czyli identyczne wartości na obu kościach, odblokowuje budynek specjalny w postaci placu (**plaza**).

4.3 Typy budynków i algorytm punktacji

System budynków w grze obejmuje trzy typy podstawowe oraz dwa typy specjalne. Budynki podstawowe to dom (**house**), las (**forest**) oraz jezioro (**lake**), dostępne w standardowych rzutach kośćmi. Budynki specjalne, fabryka (**factory**) i plac (**plaza**), dostępne są wyłącznie przy określonych kombinacjach rzutów opisanych w poprzedniej sekcji.

Algorytm obliczania punktów, zaimplementowany w metodzie prywatnej `calculateScore()`, analizuje połączenia między komórkami planszy. Podstawowe punkty przyznawane są za każdą komórkę zawierającą budynek, która jest połączona z aktualnie punktowanym wierszem. Połączenie definiowane jest jako ciąg sąsiadujących komórek (w kierunkach horyzontalnym i wertykalnym) tego samego typu budynku.

Analiza połączeń między komórkami odbywa się w sposób rekurencyjny. Dla każdej komórki na planszy uruchamiamy funkcję, która sprawdza, czy obecna komórka jest w punktowanym wierszu. Jeśli tak, zwracamy prawdę i podliczamy punkty z tej komórki. W przeciwnym wypadku, rekurencyjnie uruchamiamy sprawdzanie, czy którakolwiek z sąsiednich komórek spełnia ten warunek. Aby uniknąć zapętlenia, zapisujemy dodatkowo sprawdzone już komórki i ignorujemy je w kolejnych wywołaniach.

Fabryki generują dodatkowe punkty w zależności od sąsiedztwa. Każda fabryka sąsiadująca z lasem lub jeziorem otrzymuje bonus 10 punktów, natomiast sąsiedztwo z domem skutkuje karą 2 punktów, a sąsiedztwo z placem karą 5 punktów. Place generują znaczący bonus 10 punktów, gdy w ich bezpośrednim sąsiedztwie znajdują się jednocześnie dom, las i jezioro.

4.4 Mechanizm cofania i ponawiania akcji

System Undo został zaimplementowany w oparciu o wzorzec Memento, wykorzystujący snapshoty stanu gry. Typ `TurnSnapshot` definiuje strukturę migawki, obejmującą stan planszy (`board`), pozostałe do wykonania umieszczenia budynków (`remainingPlacements`), flagę pierwszego postawionego budynku (`isFirstBuildingPlaced`), zbiór wykorzystanych budynków bonusowych (`usedBonusBuildings`) oraz wybrany wiersz do punktacji (`selectedRow`).

Przed każdą akcją gracza wywoływana jest metoda `saveSnapshot()`, która tworzy głęboką kopię bieżącego stanu i zapisuje ją na stosie stanów poprzednich. Metoda `undo()` przywraca ostatni zapisany stan ze stosu, jednocześnie zapisując bieżący stan na stosie stanów do ponowienia.

Implementacja zapewnia, że gracz może wielokrotnie cofać swoje decyzje w ramach pojedynczej tury, co jest istotne ze względu na strategiczny charakter gry, gdzie optymalne rozmieszczenie budynków wymaga eksperymentowania z różnymi konfiguracjami.

5 Komponenty interfejsu użytkownika

5.1 Strona główna i formularz inicjalizacji

Strona główna aplikacji, zdefiniowana w pliku `src/app/page.tsx`, prezentuje użytkownikowi formularz inicjalizacji gry stylizowany na wniosek o pozwolenie na budowę. Wybór tej

konwencji wizualnej nawiązuje do tematyki gry, w której gracz wciela się w rolę architekta planującego zabudowę miasta.

Tło strony wykorzystuje charakterystyczny wzór papieru kancelaryjnego, zaimplementowany przy użyciu gradientów CSS. Klasa `bg-grid-pattern` definiuje nakładające się linie poziome i wertykalne w kolorze niebieskim z przezroczystością, tworzące siatkę o wymiarach 25 na 25 pikseli. Efekt ten uzupełniają elementy dekoracyjne, takie jak kubek kawy, linijka, ołówki oraz cyrkiel, renderowane przez komponent `Decorations`.

Formularz zawiera dwa pola tekstowe: imię architekta oraz nazwę projektowanego miasta. Po wypełnieniu formularza i kliknięciu przycisku zatwierdzenia, użytkownik przekierowywany jest do widoku gry z przekazanymi parametrami. Stylizacja formularza imituje oficjalny dokument z polami do wypełnienia oraz nagłówkiem.

5.2 Widok gry i komponenty planszy

Główny widok gry renderowany jest przez komponent zdefiniowany w pliku `src/app/play/single/page`. Komponent ten inicjalizuje instancję klasy `RollingVillage`, uruchamia pętlę gry oraz organizuje układ wszystkich elementów interfejsu. Layout widoku dzieli ekran na trzy sekcje: panel boczny z dostępnymi budynkami po lewej stronie, centralną planszę gry oraz panel punktacji po prawej stronie.

Komponent `Board.tsx` odpowiada za renderowanie siatki gry o wymiarach 6 kolumn na 5 wierszy. Każda komórka siatki reprezentowana jest przez komponent `Cell.tsx`, który wyświetla aktualny stan komórki (pusty, zajęty przez budynek określonego typu) oraz obsługuje interakcje użytkownika. Kliknięcie na pustą komórkę w dozwolonym wierszu wywołuje menu wyboru budynku, renderowane przez komponent `SelectBuildingMenu.tsx`.

Komponent `RemainingBuildings.tsx` prezentuje listę budynków dostępnych do umieszczenia w bieżącej turze, wizualizując wynik rzutu kośćmi. Po zakończeniu gry komponent ten zmienia swoją funkcję, wyświetlając podsumowanie rozgrywki wraz z końcowym wynikiem punktowym.

Komponent `Score.tsx` prezentuje bieżącą punktację gracza oraz informacje kontekstowe o stanie gry, takie jak numer rundy, aktualną fazę oraz instrukcje dla gracza. Interfejs zawiera również przyciski akcji: zatwierdzenie tury (`confirmTurn`) i cofnięcie ostatniej akcji (`undo`).

5.3 Responsywność interfejsu

Aplikacja została zaprojektowana z myślą o responsywności, zapewniając poprawne wyświetlanie zarówno na urządzeniach mobilnych, jak i desktopowych. Implementacja responsywności opiera się na dwóch mechanizmach: dynamicznym skalowaniu CSS oraz adaptacyjnych komponentach dialogowych.

Dynamiczne skalowanie zaimplementowano przy użyciu media queries reagujących na wysokość okna przeglądarki. Klasa `zoom-container` stosuje transformację `scale()` z wartościami malejącymi wraz ze zmniejszaniem się dostępnej wysokości ekranu. Dla ekranów o wysokości poniżej 1000 pikseli stosowana jest skala 0.90, dla wysokości poniżej 900 pikseli skala 0.80, a dla jeszcze mniejszych ekranów odpowiednio niższe wartości. Rozwiązanie to zapewnia, że cała plansza gry pozostaje widoczna bez konieczności przewijania.

Na urządzeniach mobilnych panele boczne (dostępne budynki oraz punktacja) zostają ukryte z głównego układu i udostępnione poprzez modalne okna dialogowe. Wykorzystano w tym celu komponent `Dialog` z biblioteki Radix UI, dostosowany wizualnie do stylistyki

aplikacji. Przyciski otwierające dialogi umieszczono w górnej części ekranu, zapewniając łatwy dostęp do wszystkich funkcji gry.

5.4 Komponenty biblioteki shadcn/ui

Generyczne komponenty interfejsu użytkownika pochodzą z systemu shadcn/ui i zostały umieszczone w katalogu `src/components/ui/`. Komponent `Button` stanowi podstawowy element interaktywny, wykorzystywany do wszystkich przycisków w aplikacji. Implementacja opiera się na bibliotece `class-variance-authority`, definiując siedem wariantów wizualnych oraz trzy rozmiary.

Komponent `Dialog` wykorzystywany jest do wyświetlania modalnych okien dialogowych, w szczególności na urządzeniach mobilnych. Customowa wersja komponentu, nazwana `DialogSide`, rozszerza standardową funkcjonalność o możliwość wyświetlania dialogu wysuwanego z boku ekranu, co lepiej odpowiada wzorcom nawigacyjnym na urządzeniach dotykowych.

Funkcja pomocnicza `cn()`, zdefiniowana w pliku `src/lib/utils.ts`, stanowi kluczowy element integracji Tailwind CSS z komponentami. Funkcja ta łączy biblioteki `clsx` i `tailwind-merge`, umożliwiając bezpieczne łączenie klas CSS z automatycznym rozwiązywaniem konfliktów między klasami Tailwind.

6 Problemy i rozwiązania

6.1 Reaktywność stanu gry w kontekście React

Jednym z głównych wyzwań technicznych napotkanych podczas implementacji była integracja mutowalnej klasy silnika gry z reaktywnym modelem React. Klasa `RollingVillage` przechowuje stan gry jako pola instancji, które są modyfikowane bezpośrednio przez metody klasy. Model ten, choć naturalny z perspektywy programowania obiektowego, stoi w sprzeczności z paradygmatem React, który oczekuje niemutowalnych aktualizacji stanu do wykrywania zmian wymagających ponownego renderowania.

Rozwiązaniem tego problemu stało się wykorzystanie hooka `useReducer` w niekonwencjonalny sposób. Zamiast definiować reducer zarządzający stanem, utworzono prosty mechanizm wymuszający ponowne renderowanie komponentu. Wyrażenie `const [, forceRerender] = useReducer(x => x + 1, 0)` tworzy funkcję `forceRerender`, której każde wywołanie inkrementuje wewnętrzny licznik, powodując ponowne renderowanie komponentu.

Funkcja `forceRerender()` wywoływana jest po każdej akcji modyfikującej stan gry, takiej jak umieszczenie budynku czy zatwierdzenie tury. Podejście to, choć niestandardowe, okazało się skuteczne w kontekście aplikacji, gdzie pojedyncza instancja klasy gry stanowi źródło prawdy dla całego stanu rozgrywki.

6.2 Implementacja pętli gry z wykorzystaniem `requestAnimationFrame`

Mechanika gry `Rolling Village` wymaga ciągłego monitorowania stanu i automatycznego przeprowadzania przejść między fazami. Wyzwanie polegało na zaimplementowaniu pętli gry (game loop), która byłaby wywoływana synchronicznie z odświeżaniem ekranu, jednocześnie integrując się z cyklem życia komponentów React.

Rozwiązanie oparto na funkcji `requestAnimationFrame`, która planuje wykonanie callback'a przed następnym odmalowaniem ekranu przez przeglądarkę. Pętla gry została zaimplementowana jako rekurencyjna funkcja `loop()`, która wywołuje metodę `tick()` silnika gry, sprawdza czy wymagany jest rzut kośćmi, wymusza ponowne renderowanie komponentu, a następnie planuje kolejne wywołanie samej siebie.

Inicjalizacja pętli odbywa się w hooku `useEffect` z pustą tablicą zależności, co zapewnia jednokrotne uruchomienie przy montowaniu komponentu. Funkcja czyszcząca hooka zmienia flagę `active` w celu zatrzymania pętli przy odmontowywaniu komponentu, zapobiegając wyciekowi pamięci i błędom związanym z aktualizacją stanu odmontowanego komponentu.

6.3 Algorytm wyszukiwania alternatywnych kolumn

Reguły gry Rolling Village przewidują sytuację, gdy docelowa kolumna planszy jest całkowicie zajęta. W takim przypadku gracz może umieścić budynek w najbliższej wolnej kolumnie. Implementacja tego mechanizmu wymagała opracowania algorytmu przeszukującego planszę w sposób spiralny, rozpoczynając od docelowej kolumny i naprzemiennie sprawdzając kolumny po lewej i prawej stronie.

Metoda prywatna `findAlternativeColumns()` przyjmuje numer docelowej kolumny oraz informację o już zarezerwowanych polach w tej kolumnie. Algorytm iteruje przez wszystkie kolumny planszy, obliczając dla każdej odległość od kolumny docelowej oraz liczbę dostępnych pól. Kolumny sortowane są według kryterium odległości, a w przypadku równej odległości preferowana jest kolumna z większą liczbą wolnych pól.

Wynikiem działania metody jest lista indeksów kolumn, z których gracz może wybrać lokalizację dla budynku. Zaimplementowane jest również zawijanie planszy horyzontalnie. Oznacza to, że jeśli docelowa kolumna znajduje się na skraju planszy, algorytm uwzględni również kolumny z przeciwległego końca planszy jako najbliższe alternatywy.

7 Podsumowanie

Realizacja projektu Rolling Village pozwoliła na praktyczne zastosowanie nowoczesnych technologii frontendowych w kontekście interaktywnej aplikacji webowej. Wykorzystanie frameworka Next.js 15 z App Router, biblioteki React 19 oraz języka TypeScript zapewniło solidną podstawę architektoniczną, umożliwiającą dalszy rozwój aplikacji.

Tryb gry jednoosobowej został w pełni zaimplementowany, oferując kompletne doświadczenie rozgrywki zgodne z regułami oryginalnej gry planszowej. Interfejs użytkownika, zbudowany w oparciu o komponenty `shadcn/ui` oraz framework Tailwind CSS, zapewnia estetyczny wygląd i responsywność na różnych urządzeniach. Charakterystyczna stylizacja papieru milimetrowego nadaje aplikacji unikalny charakter, nawiązujący do tematyki architektonicznej gry.

Separacja logiki gry od warstwy prezentacji, zrealizowana poprzez wyodrębnienie klasy `RollingVillage`, stanowi kluczowy element architektury, ułatwiający testowanie oraz potencjalne przeniesienie logiki do innych platform. Mechanizm Undo oraz automatyczne przejścia między fazami gry zapewniają płynne doświadczenie użytkownika.

Projekt przewiduje możliwość rozszerzenia o tryb wieloosobowy, czego odzwierciedleniem jest przygotowana struktura katalogów dla endpointów API oraz dynamicznych tras routingu. Implementacja tej funkcji stanowi naturalny kierunek dalszego rozwoju aplikacji. Dodatkowo, wprowadzenie testów jednostkowych dla klasy silnika gry oraz testów

integracyjnych dla komponentów React zwiększyłyby niezawodność aplikacji i ułatwiło jej utrzymanie.

8 Bibliografia

Bibliografia

- [1] Vercel Inc. *Next.js Documentation*. Dostępne online: <https://nextjs.org/docs>
- [2] Meta Platforms, Inc. *React Documentation*. Dostępne online: <https://react.dev>
- [3] Microsoft Corporation. *TypeScript Documentation*. Dostępne online: <https://www.typescriptlang.org/docs/>
- [4] Tailwind Labs Inc. *Tailwind CSS Documentation*. Dostępne online: <https://tailwindcss.com/docs>
- [5] shadcn. *shadcn/ui Documentation*. Dostępne online: <https://ui.shadcn.com>
- [6] WorkOS. *Radix UI Primitives*. Dostępne online: <https://www.radix-ui.com/primitives>
- [7] Mozilla Developer Network. *MDN Web Docs*. Dostępne online: <https://developer.mozilla.org>